

Why a ReAct Agent Cannot Parallelize: The Critical Path

Step t cannot start until step $t-1$ returns —
so the wall-clock is $N \times L$, and no number of cores can shrink it.

What this document does. It takes the one tiny agent run from Module 0.0 — the *Customer-Support Resolver* that goes `crm_lookup` → `shipping_status` → `final_answer` in three steps — and shows that the three steps form a *strict chain*: `shipping_status` needs the `order_id` that only `crm_lookup`'s observation provides, so it physically cannot begin early. We fix the per-step latency at $L \approx 1.4$ s, derive the critical-path latency $N \cdot L$, prove with Amdahl's law that throwing processors at it buys essentially nothing, and reproduce the module's quoted `latency=4.2s` for the three-step trace. Every number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact.

Where this sits. This is **Topic 2 of Module 0.0** — the latency twin of Topic 1 (which counts the same loop's *dollars*). Its companions: Topic 3 of this module (where the seconds actually go, step by step) and Module 0.2 (function calling from scratch — where you will meet the *one* place parallelism is legal: independent tool calls inside a single step).

Concepts covered, in order: the strict data dependency `thought`→`action`→`observation` · why step t blocks on step $t-1$ · critical-path latency $N \cdot L$ · invariance to core count · Amdahl's law $1/(s + (1-s)/p)$ · the serial fraction $s \approx 1$ · the speedup ceiling $1/s$ · what *can* parallelize (forward-ref Module 0.2).

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: a chain that refuses to be a fan

A ReAct agent runs a loop. Each turn the model (i) reads everything so far, (ii) emits a *thought* and a tool *action*, and (iii) the runtime runs the tool and appends the *observation*. The next turn's prompt *contains* that observation. That single fact — the prompt of step t literally includes the observation of step $t-1$ — is what forbids parallelism. You cannot start computing an input that does not exist yet.

The module's trace makes the dependency concrete:

```

1 # STEP 1
2 ACTION -> crm_lookup(user_id="U-9214")
3 OBSERVE -> { orders: ["ORD-7711"], tier: "gold" } # <- produces order_id
4 # STEP 2
5 ACTION -> shipping_status(order_id="ORD-7711") # <- CONSUMES it
6 OBSERVE -> { status: "delayed", eta: "+3d" }
7 # STEP 3
8 ACTION -> final_answer(answer="...refund offered...")
9 # TRAJECTORY COMPLETE steps=3 cost=$0.041 latency=4.2s

```

Step 2's argument `order_id="ORD-7711"` is a *value* that did not exist until step 1's observation returned it. No scheduler, no GPU, no amount of foresight can run `shipping_status` before `crm_lookup` finishes. The chain is the algorithm.

The quantities. We give every step the same latency L , split into the model's "decide" time

and the tool’s run time. Round numbers, chosen so the wall-clock is followable on paper.

Quantity	Symbol	Value here
LLM decide time per step (reason + emit action)	L_{llm}	1.1 s
Tool execution time per step	L_{tool}	0.3 s
Per-step latency	$L = L_{\text{llm}} + L_{\text{tool}}$	1.4 s
Steps in this run	N	3
Processors / cores available	p	any number
Serial fraction of the work	s	≈ 1

Notation. The *critical path* of a computation is its longest chain of strictly dependent operations — the part that must happen in order. Its length is the minimum possible wall-clock time, no matter how wide your hardware. We will show the agent’s critical path is the *entire* run.

Intuition

Parallelism speeds up a *fan* — many independent jobs you can hand to many workers at once. It does nothing for a *chain* — a relay race where each runner needs the baton from the one before. A ReAct loop is a relay race: every step’s prompt holds the previous step’s observation, so the baton is mandatory. Ten thousand idle runners on the sideline cannot make a relay finish faster.

2 Step 1 — the strict data dependency, written down

Let o_{t-1} be the observation returned by step $t-1$ and prompt_t the input the model reads on step t . The ReAct construction guarantees

$$\text{prompt}_t = [\text{base context}, \dots, \text{action}_{t-1}, o_{t-1}]. \quad (1)$$

Because o_{t-1} appears *inside* prompt_t , step t cannot even be *prepared* until step $t-1$ has returned. Formally, write $\text{start}(t)$ and $\text{end}(t)$ for the wall-clock start and end of step t . The dependency forces

$$\text{start}(t) \geq \text{end}(t-1), \quad \text{end}(t) = \text{start}(t) + L.$$

Unrolling from $\text{start}(1) = 0$ gives $\text{end}(t) = t \cdot L$. There is no slack: every step begins the instant the previous one ends, and not a moment sooner.

Mini-example — this operation in isolation

Take just steps 1 and 2 of the trace. Step 1 calls `crm_lookup` and, 1.4s later, returns `orders=["ORD-7711"]`. Only *now* does the string "ORD-7711" exist, and step 2’s action is `shipping_status(order_id="ORD-7711")`. So $\text{start}(2) \geq \text{end}(1) = 1.4\text{s}$. If you had launched `shipping_status` at $t = 0$ “to save time,” you would have had no order id to pass it — the call is undefined. The 1.4s is not waste; it is the time it took to *learn* the argument.

3 Step 2 — critical-path latency is $N \cdot L$

Sum the per-step latencies along the chain. With N strictly dependent steps each costing L :

$$\text{Lat}(N) = \sum_{t=1}^N L = N \cdot L. \quad (2)$$

For the Customer-Support Resolver, $N = 3$ and $L = 1.4\text{s}$:

Step	What happens	starts at	ends at	cumulative
1	crm_lookup	0.0 s	1.4 s	1.4 s
2	shipping_status	1.4 s	2.8 s	2.8 s
3	final_answer	2.8 s	4.2 s	4.2 s

Three steps land at exactly **4.2 s** — the module’s quoted `latency=4.2s`. ✓ Sample check: $N \cdot L = 3 \times 1.4 = 4.2$. The wall-clock is the chain length times the per-link cost, full stop.

Intuition

Notice what is *absent* from Equation (2): the number of cores. The formula does not contain p at all. That is the whole point — there is nothing for a second processor to do, because at every instant exactly one step is runnable and the rest are blocked waiting for its output. Latency here is a property of the *dependency graph*, not of the hardware.

4 Step 3 — adding cores changes nothing

Suppose a generous engineer gives the runtime p cores and asks it to “run the steps in parallel.” What happens? At $t = 0$ only step 1 is runnable (steps 2 and 3 are waiting for inputs that do not exist). So $p-1$ cores sit idle. At $t = 1.4\text{s}$ step 1 finishes and step 2 becomes runnable — again, exactly one. The schedule is forced:

Cores p	runnable at any instant	idle cores	wall-clock
1	1	0	4.2 s
2	1	1	4.2 s
3	1	2	4.2 s
1000	1	999	4.2 s

The wall-clock is invariant: 4.2s at one core, 4.2s at a thousand. ✓ You are not paying for compute; you are paying for *rounds*. A fully serial chain has exactly N rounds and no parallel speedup is possible.

Watch out

“Parallelize the agent” is a category error people make constantly. The steps of a *single* ReAct trajectory cannot be parallelized — they are a chain. What you *can* parallelize is across *independent trajectories* (run 100 separate support tickets at once; that is throughput,

not latency) or across independent tool calls *within one step* (see Section 6). Confusing “more cores” with “faster single answer” wastes both money and engineering time.

5 Step 4 — Amdahl’s law puts a ceiling on it

Amdahl’s law gives the maximum speedup from parallelizing a job whose work is a fraction s *serial* (un-parallelizable) and $1 - s$ parallelizable, run on p processors:

$$\text{speedup}(s, p) = \frac{1}{s + \frac{1-s}{p}} \quad (3)$$

In a ReAct loop the reasoning chain *is* the serial fraction, so $s \approx 1$. As $p \rightarrow \infty$ the term $(1-s)/p \rightarrow 0$ and the speedup tends to its ceiling $1/s$. With s near one, that ceiling is barely above one no matter how many processors you buy:

serial frac. s	$p = 2$	$p = 4$	$p = 8$	$p \rightarrow \infty$ (ceiling $1/s$)
0.95	1.0256	1.0390	1.0458	1.0526
0.99	1.0050	1.0076	1.0088	1.0101
1.00	1.0000	1.0000	1.0000	1.0000

Even at a generous $s = 0.95$ — pretending 5% of the agent’s work were magically independent — infinite processors cap the speedup at $1/0.95 = \mathbf{1.0526}$. ✓ A 5% improvement, for an unbounded hardware budget. For a true ReAct chain ($s = 1$) the speedup is exactly 1.0000: parallelism is worthless. Sample check, $s = 0.95$, $p = 4$: $1/(0.95 + 0.05/4) = 1/0.9625 = 1.0390$.

Intuition

Amdahl’s law is just Equation (2) wearing a suit. The serial fraction s measures how much of the job is a chain; the agent’s job is *all* chain, so $s = 1$ and the law collapses to “speedup = 1.” The only way to move the ceiling is to make part of the work genuinely independent — which means changing the *algorithm*, not the hardware.

6 Step 5 — the one place parallelism is legal

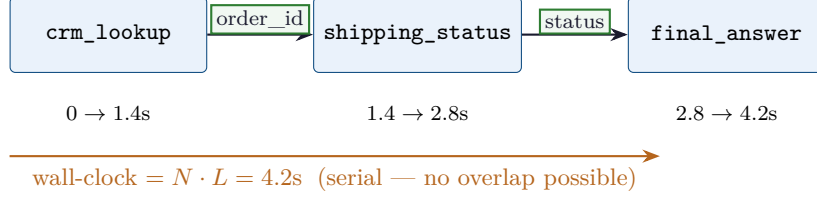
The chain is between steps. *Within* a single step, a model can sometimes emit several tool calls that do *not* depend on each other — and those can fan out. If step 1 needed both the customer’s CRM record *and* their loyalty balance, and neither argument depended on the other, the runtime could fire both at once. The step’s latency would then be the *slower* of the two tools, not their sum:

$$L_{\text{step}}^{\text{parallel}} = L_{\text{llm}} + \max_i L_{\text{tool},i} \quad \text{vs.} \quad L_{\text{step}}^{\text{serial}} = L_{\text{llm}} + \sum_i L_{\text{tool},i}.$$

But that saving lives *inside* one node of the chain; it cannot collapse the chain itself. Our trace has no such independent pair — `shipping_status` strictly needs `crm_lookup`’s output — so its

critical path stays $N \cdot L$. You build the machinery for legal intra-step parallelism in **Module 0.2**; the rule there is the same rule here, read forwards: *independent* calls may overlap, *dependent* ones may not.

7 The chain, drawn



Each arrow is a value the next step *must* have before it can run. The arrows are the critical path; their lengths sum to the wall-clock. There is no second row of boxes to run in parallel, because there is no second runnable step at any instant.

8 Critical-path cheat-sheet

Strict dependency	$\text{prompt}_t \ni o_{t-1} \Rightarrow \text{start}(t) \geq \text{end}(t-1)$
End time of step t	$\text{end}(t) = t \cdot L$
Critical-path latency	$\text{Lat}(N) = N \cdot L$ (no p term)
Per-step latency	$L = L_{\text{llm}} + L_{\text{tool}}$
Amdahl speedup	$\text{speedup} = 1/(s + (1-s)/p)$
Speedup ceiling	$\lim_{p \rightarrow \infty} \text{speedup} = 1/s$
ReAct chain	$s \approx 1 \Rightarrow \text{speedup} \approx 1$ for all p
Legal parallelism	independent tool calls <i>within</i> a step: $L_{\text{llm}} + \max_i L_{\text{tool},i}$

9 An honest word about these numbers

The per-step latency $L = 1.4\text{ s}$ (1.1 s decide + 0.3 s tool) is an illustrative round number, picked so the trace lands cleanly on the module’s 4.2 s. Real per-step latency varies wildly: a long reasoning chain or a slow external API can push L to many seconds, and it is not even constant across steps (Topic 3 derives why it *grows*). The serial fraction s is written as “ ≈ 1 ” because a real run may have a sliver of genuinely independent work; we showed even a generous $s = 0.95$ caps speedup at $1.05\times$. What is *not* illustrative is the structure: as long as step t ’s prompt contains step $t-1$ ’s observation, the steps form a strict chain, the critical-path latency is $N \cdot L$, and that quantity is mathematically independent of the processor count. Change the constants and the seconds move; the un-parallelizable shape does not. That shape is why agents feel slow, why latency budgets are about *step count*, and why “add more compute” is the wrong lever for a single trajectory.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-step schedule, the critical-path latency, the core-invariance table, and the Amdahl ceiling; change L , N , or s to explore your own agent's timing.

```

1  # react_critical_path.py -- reproduces every number in "The Critical Path".
2  L_LLM = 1.1 # LLM decide time per step, seconds
3  L_TOOL = 0.3 # tool execution time per step, seconds
4  L = L_LLM + L_TOOL # per-step latency = 1.4 s
5  N = 3 # steps in the Customer-Support Resolver trace
6
7  def critical_path(N): # Lat(N) = N * L (no dependence on cores)
8      return N * L
9
10 def amdahl(s, p): # speedup = 1 / (s + (1-s)/p)
11     return 1.0 / (s + (1.0 - s) / p)
12
13 # --- per-step schedule for the 3-step trajectory ---
14 t = 0.0
15 for step, name in enumerate(["crm_lookup", "shipping_status", "final_answer"], 1):
16     start = t; t += L
17     print(f"step {step}: {name:16s} start={start:.1f}s end={t:.1f}s")
18 print(f"critical path = N*L = {N}*{L:.1f} = {critical_path(N):.1f}s (module: 4.2s)")
19
20 # --- adding cores changes nothing: chain is fully serial ---
21 for p in (1, 2, 3, 1000):
22     print(f"cores={p:4d} runnable=1 wall-clock={critical_path(N):.1f}s")
23
24 # --- Amdahl's law: serial reasoning chain caps the speedup ---
25 for s in (0.95, 0.99, 1.0):
26     row = " ".join(f"p={p}:{amdahl(s,p):.4f}" for p in (2, 4, 8))
27     print(f"s={s:<4} {row} ceiling(1/s)={1.0/s:.4f}")
28
29 # Expected output:
30 # step 1: crm_lookup start=0.0s end=1.4s
31 # step 2: shipping_status start=1.4s end=2.8s
32 # step 3: final_answer start=2.8s end=4.2s
33 # critical path = N*L = 3*1.4 = 4.2s (module: 4.2s)
34 # cores= 1 runnable=1 wall-clock=4.2s
35 # cores= 2 runnable=1 wall-clock=4.2s
36 # cores= 3 runnable=1 wall-clock=4.2s
37 # cores=1000 runnable=1 wall-clock=4.2s
38 # s=0.95 p=2:1.0256 p=4:1.0390 p=8:1.0458 ceiling(1/s)=1.0526
39 # s=0.99 p=2:1.0050 p=4:1.0076 p=8:1.0088 ceiling(1/s)=1.0101
40 # s=1.0 p=2:1.0000 p=4:1.0000 p=8:1.0000 ceiling(1/s)=1.0000

```

— end of document —